ECE 759
High Performance Computing for Engineering Applications
Assignment 3

Submitted by: Zhengyang Wan

My GitHub link: https://github.com/BillWan-zzzyyy/repo759/tree/main/HW03

1.

In HW02 task3, several different ways to carry out matrix multiplication were implemented in sequential computing. In this task, the `mmul2` function from HW02 is parallelized with OpenMP in a function called `mmul`.

(a) Implement the function `mmul` in a file called `matmul.cpp`, the parallel version of the `mmul2` function in HW02 task3 with the prototype defined as in `matmul.h`.

Answer:

```
#include "matmul.h"

#include <algorithm>

// Parallel version of mmul2 from HW02 (loop order i, k, j).
void mmul(const float* A, const float* B, float* C, const std::size_t n) {
  // C has to start from zero since the loops only do C += A * B
  std::fill(C, C + n * n, 0.0f);

  // split the rows of C between the threads, each thread only writes its own
  // rows so there is no race on C
#pragma omp parallel for
  for (std::size_t i = 0; i < n; ++i) {
    for (std::size_t k = 0; k < n; ++k) {
      for (std::size_t j = 0; j < n; ++j) {
        C[i * n + j] += A[i * n + k] * B[k * n + j];
      }
    }
  }
}
```

(b) Write a program `task1.cpp` that creates and fills with `float` type numbers the $n \times n$ square matrices `A` and `B` (n is the first command line argument), computes $C = AB$ using the parallel implementation with `t` threads (`t` is the second command line argument), and prints the first element of `C`, the last element of `C`, and the time taken to run `mmul` in *milliseconds*.

Compile: `g++ task1.cpp matmul.cpp -Wall -O3 -std=c++17 -o task1 -fopenmp`
Run: `./task1 n t`

Answer:

```
#include <omp.h>

#include <cerrno>
#include <chrono>
#include <cstdlib>
#include <iostream>
#include <random>

#include "matmul.h"

using std::chrono::duration;
using std::chrono::high_resolution_clock;

// Returns the parsed value, or 0 if str is not a positive integer.
long parse_positive(const char* str) {
  char* end = nullptr;
  errno = 0;
  const long parsed = std::strtol(str, &end, 10);
  if (errno != 0 || end == str || *end != '\0' || parsed <= 0) {
    return 0;
  }
  return parsed;
}

int main(int argc, char* argv[]) {
  if (argc != 3) {
    std::cerr << "Usage: " << argv[0] << " n t\n";
    return 1;
  }

  // n is the matrix size, t is the number of threads
  const long n_arg = parse_positive(argv[1]);
  const long t = parse_positive(argv[2]);
  if (n_arg == 0 || t == 0 || t > 20) {
    std::cerr << "n must be a positive integer and t must be in [1, 20].\n";
    return 1;
  }
  const std::size_t n = static_cast<std::size_t>(n_arg);

  float* A = new float[n * n];
  float* B = new float[n * n];
  float* C = new float[n * n];

  // same range as HW02 task3, stored in row-major order
  std::random_device rd;
  std::mt19937 gen(rd());
  std::uniform_real_distribution<float> dist(-1.0f, 1.0f);
```

```
  for (std::size_t i = 0; i < n * n; ++i) {
    A[i] = dist(gen);
    B[i] = dist(gen);
  }

  // mmul has no thread argument, so set the number of threads here
  omp_set_num_threads(static_cast<int>(t));

  high_resolution_clock::time_point start = high_resolution_clock::now();
  mmul(A, B, C, n);
  high_resolution_clock::time_point stop = high_resolution_clock::now();
  duration<double, std::milli> elapsed = stop - start;

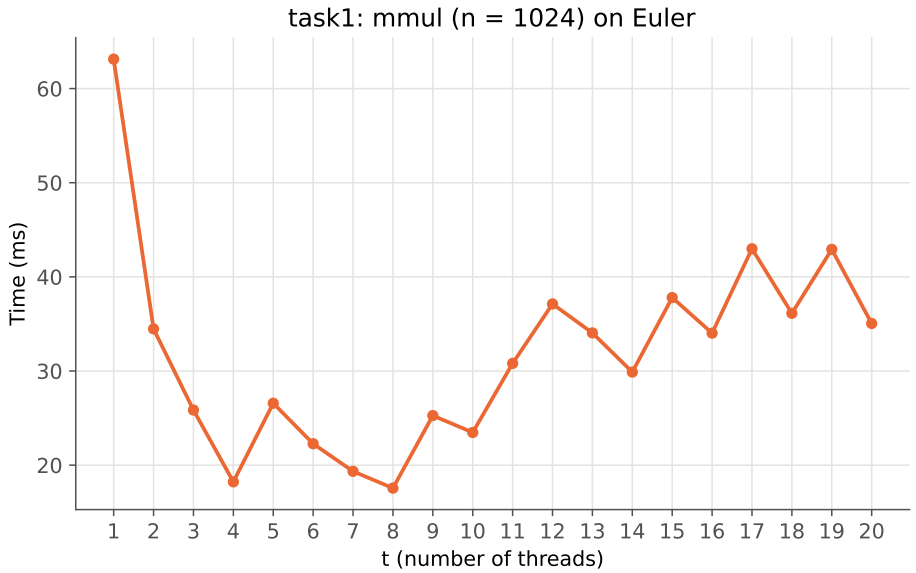
  // first and last element of C, then the time in ms
  std::cout << C[0] << "\n";
  std::cout << C[n * n - 1] << "\n";
  std::cout << elapsed.count() << "\n";

  delete[] A;
  delete[] B;
  delete[] C;

  return 0;
}
```

(c) On *Euler*, via Slurm, run `task1` for `n` $= 1024$ and `t` $= 1, 2, \cdots, 20$. Generate a plot called `task1.pdf` which plots time taken by the `mmul` function vs. `t` in linear-linear scale.

Answer: The plot is submitted separately as `task1.pdf`.



2.

In HW02 task2, the 2D convolution was implemented for sequential execution. In this task, OpenMP is used to parallelize the previous implementation.

(a) Implement in a file called `convolution.cpp` the parallel version of the `convolve` function with the prototype specified in `convolution.h`.

Answer:

```cpp
#include "convolution.h"

// Value of f[i, j] with the padding rule from HW02:
// inside the image -> image value, edges -> 1, corners -> 0.
static float get_pixel(const float *image, std::size_t n, long i, long j) {
  // use signed indices here since i or j can be negative near the border
  const long size = static_cast<long>(n);
  const bool i_in = (i >= 0 && i < size);
  const bool j_in = (j >= 0 && j < size);

  if (i_in && j_in) {
    return image[i * size + j];
  }
  // exactly one index is out of range: edge
  if (i_in || j_in) {
    return 1.0f;
  }
  // both indices are out of range: corner
  return 0.0f;
}

void convolve(const float *image, float *output, std::size_t n,
              const float *mask, std::size_t m) {
  // (m - 1) / 2 shifts the mask so that its center sits on (x, y)
  const long half = static_cast<long>((m - 1) / 2);

  // same loops as HW02, the rows of the output are split between threads.
  // every output pixel is computed independently, so no locking is needed
#pragma omp parallel for
  for (std::size_t x = 0; x < n; ++x) {
    for (std::size_t y = 0; y < n; ++y) {
      float sum = 0.0f;
      for (std::size_t i = 0; i < m; ++i) {
        for (std::size_t j = 0; j < m; ++j) {
          const long row = static_cast<long>(x + i) - half;
          const long col = static_cast<long>(y + j) - half;
          sum += mask[i * m + j] * get_pixel(image, n, row, col);
        }
      }
      output[x * n + y] = sum;
    }
```

```
  }
}
```

(b) Write a program `task2.cpp` that creates an $n \times n$ `image` matrix and a 3×3 `mask` matrix (data range and format as in HW02 task2, n is the first command line argument), applies the mask to the image using `convolve` with `t` threads (`t` is the second command line argument), and prints the first element of `output`, the last element of `output`, and the time taken to run `convolve` in *milliseconds*.
Compile: `g++ task2.cpp convolution.cpp -Wall -O3 -std=c++17 -o task2 -fopenmp`
Run: `./task2 n t`

Answer:

```
#include <omp.h>

#include <cerrno>
#include <chrono>
#include <cstdlib>
#include <iostream>
#include <random>

#include "convolution.h"

using std::chrono::duration;
using std::chrono::high_resolution_clock;

// Returns the parsed value, or 0 if str is not a positive integer.
long parse_positive(const char* str) {
  char* end = nullptr;
  errno = 0;
  const long parsed = std::strtol(str, &end, 10);
  if (errno != 0 || end == str || *end != '\0' || parsed <= 0) {
    return 0;
  }
  return parsed;
}

int main(int argc, char* argv[]) {
  if (argc != 3) {
    std::cerr << "Usage: " << argv[0] << " n t\n";
    return 1;
  }

  // n is the image size, t is the number of threads
  const long n_arg = parse_positive(argv[1]);
  const long t = parse_positive(argv[2]);
  if (n_arg == 0 || t == 0 || t > 20) {
    std::cerr << "n must be a positive integer and t must be in [1, 20].\n";
    return 1;
  }
```

```
  const std::size_t n = static_cast<std::size_t>(n_arg);
  // the mask is always 3 x 3 in this task
  const std::size_t m = 3;

  // both matrices are stored in 1D in row-major order
  float* image = new float[n * n];
  float* mask = new float[m * m];
  float* output = new float[n * n];

  // image values in [-10, 10], mask values in [-1, 1]
  std::random_device rd;
  std::mt19937 gen(rd());
  std::uniform_real_distribution<float> image_dist(-10.0f, 10.0f);
  std::uniform_real_distribution<float> mask_dist(-1.0f, 1.0f);

  for (std::size_t i = 0; i < n * n; ++i) {
    image[i] = image_dist(gen);
  }
  for (std::size_t i = 0; i < m * m; ++i) {
    mask[i] = mask_dist(gen);
  }

  // convolve has no thread argument, so set the number of threads here
  omp_set_num_threads(static_cast<int>(t));

  high_resolution_clock::time_point start = high_resolution_clock::now();
  convolve(image, output, n, mask, m);
  high_resolution_clock::time_point stop = high_resolution_clock::now();
  duration<double, std::milli> elapsed = stop - start;

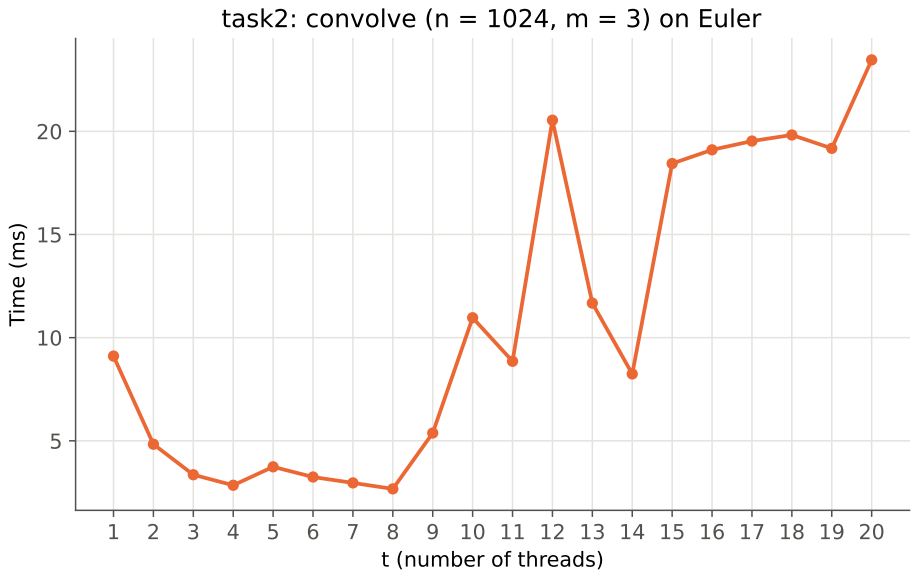
  // first and last element of the result, then the time in ms
  std::cout << output[0] << "\n";
  std::cout << output[n * n - 1] << "\n";
  std::cout << elapsed.count() << "\n";

  delete[] image;
  delete[] mask;
  delete[] output;

  return 0;
}
```

(c) On *Euler* via Slurm, run `task2` for `n` $= 1024$ and `t` $= 1, 2, \cdots, 20$. Generate a figure called `task2.pdf` which plots the time taken by the `convolve` function vs. `t` in linear-linear scale. Discuss your observations from the plot. Explain to what extent the increase in the number of threads improves the performance, and why the run time does not show significant decrease after reaching a certain number of threads.

Answer: The plot is submitted separately as `task2.pdf`.



Going from 1 to 8 threads the time drops from 9.11 ms to 2.67 ms, a speedup of about 3.4×, and most of this gain is already there at $t = 3$ or 4 (2.85 ms at $t = 4$). After about 8 threads, adding more threads does not help any more: the time goes back up and is around 18–23 ms for $t \geq 15$, which is even slower than the single-thread run.

The main reason is that the problem is small. With $n = 1024$ and a 3×3 mask the whole convolution is only about 9.4 million multiply-adds and takes about 9 ms on one thread, so with many threads each thread only gets around 50 rows, i.e. well under a millisecond of work. At that point the fixed costs dominate: `convolve` is the first parallel region in the program, so the measured time also includes starting the OpenMP threads, and every extra thread adds more creation, scheduling and waiting at the implicit barrier at the end of the `parallel for`. The loop also does very little computation per byte loaded (each output pixel reads a 3×3 neighborhood and writes one float), so once a few threads are running the speed is limited by memory bandwidth rather than by the number of cores. Finally, the Euler nodes are shared and each t was run only once, so the timings for larger t are also noisy (for example the jumps at $t = 10$ and $t = 12$).

3.

Implement a parallel merge sort using OpenMP **tasks**.

(a) Implement in a file called `msort.cpp` the parallel merge sort algorithm with the prototype specified in `msort.h`. The `msort` function takes an array of integers and returns the sorted results in place in ascending order.

Answer:

```cpp
#include "msort.h"

#include <algorithm>

// Serial sort used when the piece is smaller than the threshold.
// Insertion sort has no recursion, which is what the task asks for.
static void insertion_sort(int* arr, std::size_t n) {
  for (std::size_t i = 1; i < n; ++i) {
    const int key = arr[i];
    std::size_t j = i;
    // shift the bigger elements one step to the right
    while (j > 0 && arr[j - 1] > key) {
      arr[j] = arr[j - 1];
      --j;
    }
    arr[j] = key;
  }
}

// Merges the sorted halves arr[0, mid) and arr[mid, n) into tmp and copies
// the result back into arr.
static void merge(int* arr, int* tmp, std::size_t mid, std::size_t n) {
  std::size_t i = 0;
  std::size_t j = mid;
  std::size_t k = 0;
  while (i < mid && j < n) {
    // <= keeps equal elements in their original order
    if (arr[i] <= arr[j]) {
      tmp[k++] = arr[i++];
    } else {
      tmp[k++] = arr[j++];
    }
  }
  while (i < mid) {
    tmp[k++] = arr[i++];
  }
  while (j < n) {
    tmp[k++] = arr[j++];
  }
  std::copy(tmp, tmp + n, arr);
}

// Recursive part of the merge sort. tmp is the matching part of the
// temporary buffer, so the two halves never touch the same memory.
static void msort_rec(int* arr, int* tmp, std::size_t n,
                      std::size_t threshold) {
  if (n < 2) {
    return;
  }
  // small piece: not worth creating more tasks
  if (n < threshold) {
```

```
    insertion_sort(arr, n);
    return;
  }

  const std::size_t mid = n / 2;
  // sort the two halves as two OpenMP tasks
#pragma omp task
  msort_rec(arr, tmp, mid, threshold);
#pragma omp task
  msort_rec(arr + mid, tmp + mid, n - mid, threshold);
  // both halves have to be sorted before merging
#pragma omp taskwait
  merge(arr, tmp, mid, n);
}

void msort(int* arr, const std::size_t n, const std::size_t threshold) {
  // one buffer for all merges, allocated once
  int* tmp = new int[n];

  // create the thread team, then let one thread start the recursion; the
  // tasks it creates are picked up by the other threads
#pragma omp parallel
  {
#pragma omp single
    msort_rec(arr, tmp, n, threshold);
  }

  delete[] tmp;
}
```

(b) Write a program `task3.cpp` that creates an array `arr` of length `n` filled with random `int` numbers in the range $[-1000, 1000]$ (`n` is the first command line argument), applies `msort` to `arr` with `t` threads (`t` is the second command line argument) and threshold `ts` (third command line argument; below this size a serial sort without recursion is used), and prints the first element of `arr`, the last element of `arr`, and the time taken to run `msort` in *milliseconds*.
Compile: `g++ task3.cpp msort.cpp -Wall -O3 -std=c++17 -o task3 -fopenmp`
Run: `./task3 n t ts`

Answer:

```
#include <omp.h>

#include <cerrno>
#include <chrono>
#include <cstdlib>
#include <iostream>
#include <random>

#include "msort.h"
```

```
using std::chrono::duration;
using std::chrono::high_resolution_clock;

// Returns the parsed value, or 0 if str is not a positive integer.
long parse_positive(const char* str) {
  char* end = nullptr;
  errno = 0;
  const long parsed = std::strtol(str, &end, 10);
  if (errno != 0 || end == str || *end != '\0' || parsed <= 0) {
    return 0;
  }
  return parsed;
}

int main(int argc, char* argv[]) {
  if (argc != 4) {
    std::cerr << "Usage: " << argv[0] << " n t ts\n";
    return 1;
  }

  // n is the array length, t the number of threads, ts the threshold
  const long n_arg = parse_positive(argv[1]);
  const long t = parse_positive(argv[2]);
  const long ts_arg = parse_positive(argv[3]);
  if (n_arg == 0 || t == 0 || t > 20 || ts_arg == 0) {
    std::cerr << "n and ts must be positive integers and t must be in "
                 "[1, 20].\n";
    return 1;
  }
  const std::size_t n = static_cast<std::size_t>(n_arg);
  const std::size_t ts = static_cast<std::size_t>(ts_arg);

  // random integers in [-1000, 1000]
  int* arr = new int[n];
  std::random_device rd;
  std::mt19937 gen(rd());
  std::uniform_int_distribution<int> dist(-1000, 1000);
  for (std::size_t i = 0; i < n; ++i) {
    arr[i] = dist(gen);
  }

  // msort has no thread argument, so set the number of threads here
  omp_set_num_threads(static_cast<int>(t));

  high_resolution_clock::time_point start = high_resolution_clock::now();
  msort(arr, n, ts);
  high_resolution_clock::time_point stop = high_resolution_clock::now();
  duration<double, std::milli> elapsed = stop - start;

  // first and last element of the sorted array, then the time in ms
```

```
  std::cout << arr[0] << "\n";
  std::cout << arr[n - 1] << "\n";
  std::cout << elapsed.count() << "\n";

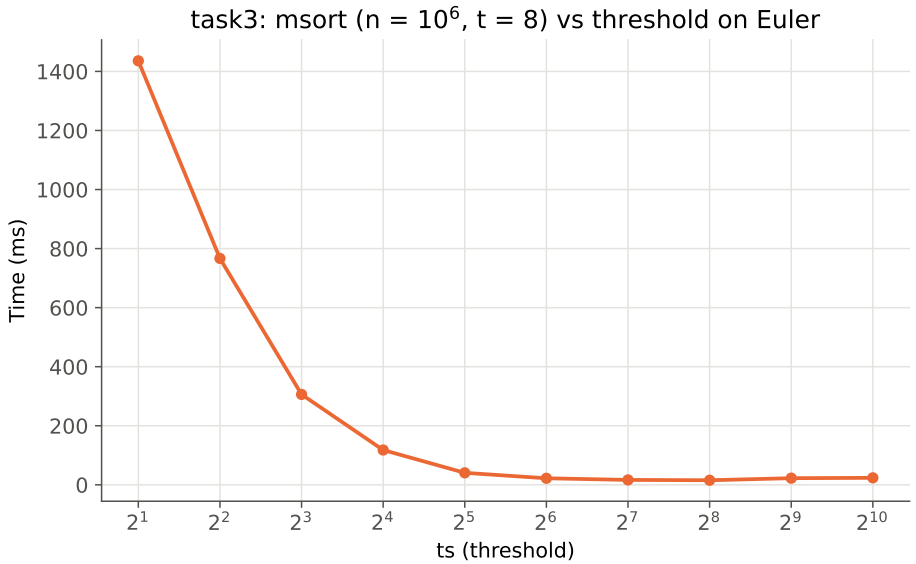
  delete[] arr;

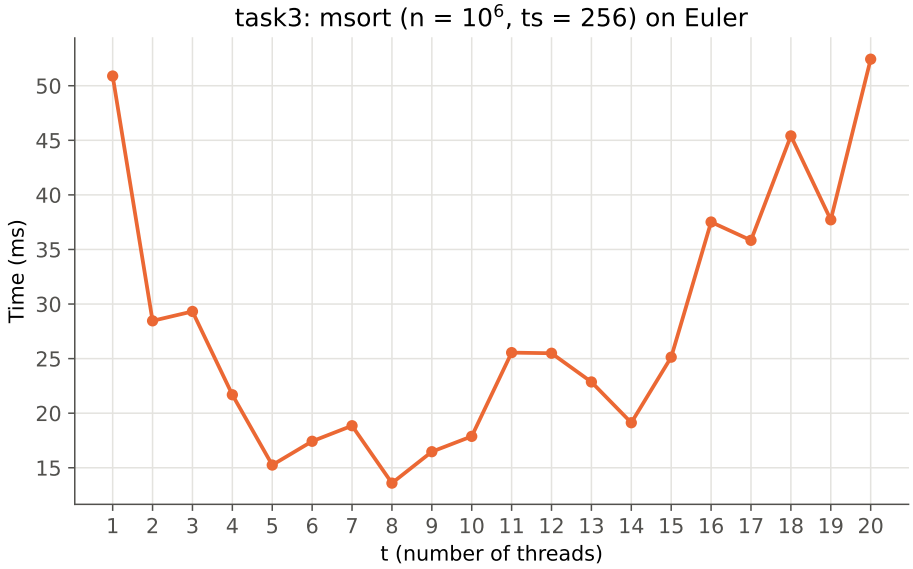
  return 0;
}
```

(c) On *Euler* via Slurm:

- Run `task3` for `n` $= 10^6$, `t` $= 8$, and `ts` $= 2^1, 2^2, \cdots, 2^{10}$. Generate a plot called `task3_ts.pdf` which plots the time taken by the `msort` function vs. `ts` in linear-log scale.
- Run `task3` for `n` $= 10^6$, `t` $= 1, 2, \cdots, 20$, and `ts` equal to the value that yields the best performance in the plot of time vs. `ts`. Generate a plot called `task3_t.pdf` which plots time taken by the `msort` function vs. `t` in linear-linear scale.

Answer: The plots are submitted separately as `task3_ts.pdf` and `task3_t.pdf`. In the threshold sweep with $t = 8$ the fastest value was `ts` $= 256$ (15.5 ms), so `ts` $= 256$ is used for the sweep over t.




task3: msort (n = 10^6, ts = 256) on Euler
Time (ms)
50
45
40
35
30
25
20
15
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
t (number of threads)